

COMPLETE REFERENCE NOTES

Mathematics for AI Engineers

Linear Algebra | Calculus | Probability | Statistics
Optimization | Information Theory | Numerical Methods

Prepared for Sameeksha

TABLE OF CONTENTS

01 Linear Algebra	02 Calculus & Derivatives
03 Multivariable Calculus	04 Probability Theory
05 Statistics	06 Optimization
07 Information Theory	08 Numerical Methods & Tricks

Linear Algebra

Scalars, Vectors, Matrices & Tensors

SCALAR

A single number x in $\mathbb{R}$. Used for learning rate, loss value, weights.

VECTOR

Ordered array v in $\mathbb{R}^n$. Represents features, embeddings, or parameters.

MATRIX

2D array A in $\mathbb{R}^{(m \times n)}$. Represents weight layers in neural networks.

TENSOR

N -dimensional generalization. Images are rank-3 tensors ($H \times W \times C$).

Vector Operations

Dot Product: $a \cdot b = \sum (a_i \cdot b_i) = |a| |b| \cos(\theta)$

L2 Norm: $\|x\|_2 = \sqrt{\sum x_i^2}$

L1 Norm: $\|x\|_1 = \sum |x_i|$

Cosine Sim: $\cos(\theta) = (a \cdot b) / (|a| \cdot |b|)$

AI Use: Cosine similarity powers word embeddings, semantic search, and recommendation systems. It measures angle between vectors, not magnitude.

Matrix Operations

Matrix Mult: $C = AB$, $C_{ij} = \sum_k A_{ik} B_{kj}$

Transpose: $(A^T)_{ij} = A_{ji}$

Inverse: $A \cdot A^{-1} = I$ (square, non-singular only)

Hadamard: $(A \circ B)_{ij} = A_{ij} \cdot B_{ij}$ (element-wise product)

Neural net forward pass: $z = W \cdot x + b$

Attention: $\text{Attn}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) \cdot V$

Eigenvalues & Eigenvectors

Definition: $A \cdot v = \lambda \cdot v$

Characteristic eq: $\det(A - \lambda I) = 0$

Eigendecomp: $A = Q \Lambda Q^{-1}$

Symmetric: $A = Q \Lambda Q^T$ (Q orthogonal)

PCA finds eigenvectors of the covariance matrix. The largest eigenvalue corresponds to the direction of maximum variance — used for dimensionality reduction.

Singular Value Decomposition (SVD)

$A = U \cdot \Sigma \cdot V^T$

U in $\mathbb{R}^{(m \times m)}$ — left singular vectors (orthogonal)

Σ in $\mathbb{R}^{(m \times n)}$ — diagonal, singular values $\sigma_1 \geq \sigma_2 \geq 0$

V^T in $\mathbb{R}^{(n \times n)}$ — right singular vectors (orthogonal)

Low-rank approx: $A_k = \sum_{i=1}^k \sigma_i \cdot u_i \cdot v_i^T$

Truncated SVD powers recommender systems (matrix factorization), NLP (LSA), and data compression. It is the backbone of collaborative filtering.

Special Matrices

Matrix Type	Property	AI Use Case
Identity (I)	$I_{ij} = 1$ if $i=j$, else 0	Residual connections
Symmetric	$A = A^T$	Covariance, kernel matrices
Orthogonal	$A^T \cdot A = I$	Rotations in embedding space
Positive Definite	$x^T A x > 0$ for all $x \neq 0$	Valid covariance, convex optimization
Sparse	Mostly zeros	Graph adjacency, one-hot vectors

Calculus & Derivatives

Derivative Rules

Power Rule: $d/dx[x^n] = n \cdot x^{(n-1)}$

Product Rule: $d/dx[f \cdot g] = f' \cdot g + f \cdot g'$

Chain Rule: $d/dx[f(g(x))] = f'(g(x)) \cdot g'(x)$

Exponential: $d/dx[e^x] = e^x$

Log: $d/dx[\ln x] = 1/x$

Sigmoid: $\text{sigma}'(x) = \text{sigma}(x) \cdot (1 - \text{sigma}(x))$

Tanh: $\text{tanh}'(x) = 1 - \text{tanh}^2(x)$

ReLU: $f'(x) = 1$ if $x > 0$, else 0

Chain Rule IS Backpropagation. Every layer gradient is computed by chaining local derivatives: $dL/dw = (dL/da) \cdot (da/dz) \cdot (dz/dw)$

Activation Function Derivatives

Activation	Formula	Derivative	AI Notes
Sigmoid	$1/(1+e^{-x})$	$\text{sigma}(x)(1-\text{sigma}(x))$	Vanishing gradient issue
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	$1 - \text{tanh}^2(x)$	Zero-centered
ReLU	$\max(0, x)$	0 or 1	Fast; dying neuron problem
Leaky ReLU	$\max(ax, x)$	a or 1	Fixes dying ReLU
GELU	$x \cdot \Phi(x)$	$\Phi(x) + x \cdot \phi(x)$	Used in BERT, GPT
Softmax	$e^{x_i} / \sum(e^{x_j})$	$\pi_i(\delta_{ij} - \pi_j)$	Output probabilities

Taylor Series

$f(x) \approx f(a) + f'(a)(x-a) + f''(a)(x-a)^2/2! + \dots$

$e^x = 1 + x + x^2/2! + x^3/3! + \dots$

$\ln(1+x) \approx x - x^2/2 + x^3/3 - \dots$ for $|x| < 1$

Taylor expansion justifies gradient descent as a first-order approximation of the loss landscape. Newton methods (second-order) use the Hessian for faster convergence.

Multivariable Calculus

Gradient

Partial: df/dx_i (rate of change w.r.t. x_i , others held constant)

Gradient: $\text{grad}_f(x) = [df/dx_1, df/dx_2, \dots, df/dx_n]^T$

grad_f points in direction of steepest ASCENT

Gradient Descent: $w \leftarrow w - \eta * \text{grad}_w L$

Jacobian Matrix

For $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$

$J \in \mathbb{R}^{(m \times n)}$, $J_{ij} = df_i/dx_j$

Backprop = multiplying Jacobians across layers

$dL/dx = J^T * dL/df$

Hessian Matrix

$H \in \mathbb{R}^{(n \times n)}$, $H_{ij} = d^2f / (dx_i dx_j)$

H positive definite $\rightarrow$ local minimum

H negative definite $\rightarrow$ local maximum

Mixed signs $\rightarrow$ saddle point

Saddle points are common in deep learning. SGD with momentum escapes them efficiently. The Hessian eigenspectrum explains why some directions learn faster than others.

Backpropagation Matrix Form

For $z = W*x + b$:

$dL/dW = \text{delta}^T * x^T$ (outer product)

$dL/dx = W^T * \text{delta}$

$dL/db = \text{delta}$

where $\text{delta} = dL/dz$ (upstream gradient)

Probability Theory

Fundamentals

Complement: $P(A^c) = 1 - P(A)$

Union: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Intersection: $P(A \cap B) = P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$

Independence: $P(A \cap B) = P(A) \cdot P(B)$

Bayes' Theorem

$P(A|B) = P(B|A) \cdot P(A) / P(B)$

Full form (Law of Total Probability):

$P(B) = \sum_k P(B|A_k) \cdot P(A_k)$

Posterior = (Likelihood x Prior) / Evidence

Core of Bayesian ML, Naive Bayes classifiers, and probabilistic graphical models. GPT predicts $P(\text{next token} | \text{context})$ — that is conditional probability.

Key Probability Distributions

Distribution	PDF / PMF	Mean	Variance	AI Use
Bernoulli(p)	$p^x(1-p)^{(1-x)}$	p	$p(1-p)$	Binary classification
Gaussian $N(\mu, \sigma^2)$	$(1/\sigma\sqrt{2\pi}) \cdot e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2	Weight init, VAE latent
Categorical(p)	$\prod p_i^{x_i}$	—	—	Softmax output
Uniform $U(a,b)$	$1/(b-a)$	$(a+b)/2$	$(b-a)^2/12$	Random initialization
Laplace(μ, b)	$(1/2b) \cdot e^{- x-\mu /b}$	μ	$2b^2$	L1 regularization prior

Expectation & Variance

$E[X] = \sum x \cdot P(X=x)$ or $\int x \cdot f(x) dx$

$E[aX + b] = a \cdot E[X] + b$

$E[X + Y] = E[X] + E[Y]$ (linearity, always)

$\text{Var}[X] = E[X^2] - (E[X])^2$

$$\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)]$$

Correlation: $\rho = \text{Cov}(X, Y) / (\sigma_x * \sigma_y)$ in $[-1, 1]$

Important Theorems

LAW OF LARGE NUMBERS

As $n \rightarrow \infty$, sample mean $\bar{x} \rightarrow$ true mean μ . Foundation of Monte Carlo estimation methods.

CENTRAL LIMIT THEOREM

Sum of many independent RVs $\rightarrow$ Normal distribution regardless of individual distributions. Justifies Gaussian noise assumptions in ML.

Statistics

Maximum Likelihood Estimation (MLE)

```
theta_MLE = argmax_theta P(D|theta)

= argmax_theta sum_i log P(xi | theta)
```

For Gaussian:

```
mu_MLE = (1/n) * sum xi

sigma^2_MLE = (1/n) * sum (xi - mu)^2
```

Training a neural network with cross-entropy loss IS doing MLE. The loss function equals the negative log-likelihood of the data under the model.

Maximum A Posteriori (MAP)

```
theta_MAP = argmax_theta [log P(D|theta) + log P(theta)]

Gaussian prior -> L2 regularization (Ridge): lambda * ||w||^2

Laplace prior -> L1 regularization (Lasso): lambda * ||w||_1
```

Regularization = MAP estimation. The regularization penalty term is the negative log-prior on the weights.

Bias-Variance Tradeoff

Expected Test Error = Bias² + Variance + Irreducible Noise

Bias: Error from overly simple assumptions (underfitting)

Variance: Sensitivity to training data (overfitting)

Noise: Irreducible error inherent in data

Model	Bias	Variance	Remedy
Too simple (linear on nonlinear data)	High	Low	More capacity, more features
Too complex (deep net on small data)	Low	High	Regularization, more data
Optimal	Low	Low	Balanced model selection

Optimization

Gradient Descent Variants

```

Batch GD:  $w \leftarrow w - \eta * \text{grad}_w L(w; D)$ 

Stochastic:  $w \leftarrow w - \eta * \text{grad}_w L(w; x_i, y_i)$ 

Mini-batch:  $w \leftarrow w - \eta * \text{grad}_w L(w; B) \quad |B| \ll n$ 

Momentum:  $v \leftarrow \beta * v + \text{grad}_L$ 

 $w \leftarrow w - \eta * v$ 

Nesterov:  $v \leftarrow \beta * v + \text{grad}_L(w - \beta * v)$ 

```

Adam Optimizer (Full)

```

 $m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$  (first moment)

 $v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2$  (second moment)

 $\hat{m}_t = m_t / (1 - \beta_1^t)$  (bias correction)

 $\hat{v}_t = v_t / (1 - \beta_2^t)$  (bias correction)

 $w_t = w_{t-1} - \eta * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ 

Typical:  $\beta_1=0.9, \beta_2=0.999, \epsilon=1e-8$ 

```

Optimizer	Key Feature	When to Use
SGD + Momentum	Accumulates velocity	Simple tasks, CNNs
AdaGrad	Per-parameter LR decay	Sparse features / NLP
RMSProp	Exp. moving avg of squared grad	RNNs, online learning
Adam	Momentum + RMSProp + bias correction	Default for deep learning
AdamW	Decoupled weight decay	Transformers (BERT, GPT)

Regularization

```

L2 (Ridge):  $L_{\text{reg}} = L + \lambda * ||w||_2^2$ 

L1 (Lasso):  $L_{\text{reg}} = L + \lambda * ||w||_1$  (induces sparsity)

Dropout: Randomly zero p fraction of neurons during training

Batch Norm:  $\hat{z} = (z - \mu_B) / \sigma_B * \gamma + \beta$ 

```

Early Stop: Halt when validation loss stops improving

Information Theory

Shannon Entropy

$$H(X) = - \sum_x p(x) * \log_2 p(x) \text{ [bits]}$$

$$H(X) = - \sum_x p(x) * \ln p(x) \text{ [nats]}$$

Maximum entropy: Uniform distribution $\rightarrow H = \log(n)$

Minimum entropy: Deterministic event $\rightarrow H = 0$

Entropy measures uncertainty or information content. High entropy = more surprise when observing the outcome. Used in decision trees (information gain = entropy reduction at a split).

Cross-Entropy & KL Divergence

$$\text{Cross-Entropy: } H(p, q) = - \sum_x p(x) * \log q(x)$$

$$\text{KL Divergence: } KL(p||q) = \sum_x p(x) * \log(p(x)/q(x)) \geq 0$$

$$\text{Relation: } H(p, q) = H(p) + KL(p||q)$$

$$KL(p||q) \neq KL(q||p) \text{ (asymmetric!)}$$

$$KL = 0 \text{ if and only if } p = q$$

Cross-entropy loss for classification: $L = -\sum y_i * \log(\hat{y}_i)$. Minimizing cross-entropy is equivalent to minimizing KL divergence between true and predicted distributions.

Loss Functions & Information-Theoretic Basis

Loss Function	Formula	Use Case	Statistical Basis
Binary CE	$-[y * \log(\hat{y}) + (1-y) * \log(1-\hat{y})]$	Binary classification	MLE Bernoulli
Categorical CE	$-\sum y_i * \log(\hat{y}_i)$	Multiclass classification	MLE Categorical
MSE	$(1/n) * \sum (y - \hat{y})^2$	Regression	MLE Gaussian
MAE	$(1/n) * \sum y - \hat{y} $	Robust regression	MLE Laplace
KL Divergence	$\sum p * \log(p/q)$	VAE, knowledge distillation	Information theory

Mutual Information

$$I(X;Y) = KL(p(x,y) || p(x)*p(y))$$

$$= H(X) - H(X|Y) = H(Y) - H(Y|X)$$

$$= H(X) + H(Y) - H(X, Y)$$

$$I(X; Y) \geq 0, I(X; Y) = 0 \text{ iff } X \text{ and } Y \text{ are independent}$$

Mutual information measures how much X tells us about Y . Used in feature selection, contrastive learning (maximizing MI between augmented views), and VAEs (InfoNCE loss).

Numerical Methods & Tricks

Numerical Stability

Softmax (stable):

$$\text{softmax}(x)_i = e^{(x_i - \max(x))} / \sum_j e^{(x_j - \max(x))}$$

Log-Sum-Exp trick:

$$\log \sum e^{x_i} = c + \log \sum e^{(x_i - c)}, \quad c = \max(x)$$

Always add epsilon (1e-8) before log() to avoid log(0)

Matrix Decompositions Summary

Decomposition	Form	Use in AI
LU	$A = LU$	Solving $Ax=b$ linear systems
QR	$A = QR$	Least squares, Gram-Schmidt orthogonalization
Cholesky	$A = LL^T$ (PD only)	Sampling from Gaussian distributions
Eigendecomposition	$A = Q \Lambda Q^{-1}$	PCA, spectral clustering, graph analysis
SVD	$A = U \Sigma V^T$	Recommender systems, NLP, compression

Key Inequalities for ML Theory

Cauchy-Schwarz: $||u \cdot v|| \leq ||u|| \cdot ||v||$

Jensen's: $f(E[X]) \leq E[f(X)]$ for convex f

Used to derive the ELBO in Variational Autoencoders

Triangle: $||x + y|| \leq ||x|| + ||y||$

AM-GM: $(a + b) / 2 \geq \sqrt{a \cdot b}$

Computational Complexity Reference

Operation	Complexity	Notes
Matrix multiply $A(m \times n) * B(n \times p)$	$O(mnp)$	$O(n^3)$ for square matrices
Matrix inverse $n \times n$	$O(n^3)$	Use LU factorization instead
SVD $m \times n$	$O(\min(m,n) \cdot mn)$	Truncated SVD is much faster
Self-attention (seq len n)	$O(n^2 \cdot d)$	Quadratic in sequence length

Operation	Complexity	Notes
Gradient backprop	$O(2x \text{ forward pass})$	Roughly constant factor overhead

Common Gradients Quick Reference

$$d/dW [W^T x + b] = x^T$$

$$d/dx [W^T x + b] = W^T$$

$$d/dx [x^T A x] = (A + A^T)x \rightarrow 2Ax \text{ if symmetric}$$

$$d/dx [a^T x] = a$$

$$d/dA [\text{tr}(A^T B)] = B^T$$

$$d/dx [\|x\|_2^2] = 2x$$

$$d/dx [\|Ax - b\|_2^2] = 2A^T(Ax - b)$$